

Teoria — Kompilator

Do powiedzenia:

Kompilator to program, który przekształca CAŁY kod źródłowy na kod maszynowy lub bytecode przed uruchomieniem. Tłumaczenie odbywa się jednorazowo — wynikiem jest gotowy plik wykonywalny (.exe, ELF, .o itp.).

Etapy kompilacji:

1

Analiza leksykalna (tokenizacja)

Kod dzielony na tokeny: słowa kluczowe, identyfikatory, operatory, literały. Przykład: `int x = 5;` → `[int] [x] [=] [5] [;]`

2

Analiza składniowa (parser)

Tokeny układane w drzewo składniowe (AST). Wykrywane są błędy składniowe.

3

Analiza semantyczna

Sprawdzanie typów, deklaracji zmiennych, zakresu. Wykrywa błędy sensowności (np. przypisanie stringa do inta).

4

Optymalizacja kodu pośredniego

Eliminacja martwego kodu, upraszczanie wyrażeń stałych, inlining funkcji.

5

Generowanie kodu maszynowego

Tworzony kod zrozumiały dla procesora. Linker łączy wszystkie moduły w plik wykonywalny.

Przykład — błąd kompilatora w C++ (do pokazania uczniom):

```
// Błąd składniowy - brakuje średnika
int main() {
    int x = 10    // <-- brakuje ;
    return 0;
}

// Komunikat GCC:
// error: expected ';' after expression
//      int x = 10
//              ^
```

Kompilator NIE uruchomi programu dopóki wszystkie błędy nie zostaną poprawione. To wielka zaleta — wykrywa błędy zanim program dotrze do użytkownika.

Teoria — Interpreter

Do powiedzenia:

Interpreter czyta i wykonuje kod źródłowy linia po linii, w czasie rzeczywistym, bez wcześniejszego tłumaczenia na kod maszynowy. Każde uruchomienie wymaga interpretera.

Jak działa interpreter (schemat):

Czyta linię	→	Analizuje	→	Wykonuje	→	Kolejna linia
kod źródłowy		parsowanie + semantyka		natychmiast		aż do końca

Przykład — błąd interpretera w Pythonie (do pokazania uczniom):

```
# Ten kod uruchomi się CZĘŚCIOWO!
print("Linia 1 – działa")
print("Linia 2 – działa")

x = int("abc") # ValueError tutaj!
print("Linia 5 – NIGDY nie wykona się")

# Output:
# Linia 1 – działa
# Linia 2 – działa
# ValueError: invalid literal for int() with base 10: 'abc'
```

Interpreter wykonał linie 1 i 2, zanim wykrył błąd w linii 4. To charakterystyczna cecha — błędy pojawiają się podczas działania programu.

Teoria — Debugger

Do powiedzenia:

Debugger to narzędzie umożliwiające kontrolowane wykonywanie programu — krok po kroku, z podglądem stanu zmiennych, stosu wywołań i pamięci. Pozwala znaleźć i naprawić błędy logiczne, których kompilator ani interpreter nie wykryją.

Kluczowe funkcje debuggera:



Breakpoint (punkt zatrzymania)

Program zatrzymuje się w wybranej linii kodu. Możemy sprawdzić stan programu w tym dokładnym momencie.



Step Over / Step Into / Step Out

Step Over — wykonaj linię bez wchodzenia do funkcji; Step Into — wejdź do funkcji; Step Out — wyjdź z bieżącej funkcji.



Watch Expressions (obserwowane wyrażenia)

Monitorowanie wartości zmiennych w czasie rzeczywistym — widoczne jak zmieniają się z każdym krokiem.



Call Stack (stos wywołań)

Podgląd łańcucha wywołań funkcji — widzimy, która funkcja wywołała którą.



Variables Inspector

Podgląd dokładnych wartości wszystkich zmiennych lokalnych i globalnych w danym momencie.

Print vs Debugger:

Używanie print() do szukania błędów działa, ale jest powolne i zaśmieca kod. Debugger pozwala zbadać stan programu bez modyfikowania kodu, zatrzymując go w dowolnym momencie.

Porównanie wszystkich trzech

	Kompilator	Interpreter	Debugger
Kiedy działa	Przed uruchomieniem	Podczas wykonania (runtime)	Podczas debugowania
Wynik	Plik wykonywalny (.exe, ELF...)	Bezpośrednia odpowiedź / wynik	Informacje diagnostyczne
Błędy	Wykrywa przed uruchomieniem	Pojawiają się w trakcie działania	Pomaga znaleźć każdy rodzaj
Szybkość	Gotowy program działa b. szybko	Wolniejszy od skompilowanego	Spowalnia program (celowo)
Przenośność	Skompilowany dla konkretnego OS	Interpreter na każdym OS	Zintegrowany z IDE / narzędziem
Przykłady	C, C++, Rust, Go, Fortran	Python, Ruby, Bash, PHP	GDB, pdb, Chrome DevTools, VS Code

Hybrydy — JIT i bytecode:

Java i C# kompilują kod do bytecode (.class / .dll), a JVM / CLR interpretuje go i kompiluje do kodu maszynowego tuż przed wykonaniem (JIT). JavaScript w przeglądarkach (V8) używa JIT — zaczyna jako interpreter, ale gorący kod kompiluje w locie. Python tworzy pliki .pyc (bytecode), co przyspiesza ponowne uruchomienie.

Analiza błędów — od błędu do rozwiązania

Kluczowa zasada:

Każdy błąd mówi coś konkretnego. Czytaj komunikat od początku — zawiera typ błędu, numer linii i opis. Nie ignoruj go!

Trzy rodzaje błędów:

Typ błędu	Kiedy	Kto wykrywa	Przykład	Jak naprawić
Składniowy (Syntax)	Przed uruchomieniem	Kompilator / Interpreter	Brakujący nawias, średnik, błędne słowo kluczowe	Sprawdź wskazaną linię i popraw składnię
Czasu wykonania (Runtime)	Podczas działania	Interpreter / OS	Dzielenie przez zero, brak pliku, null pointer	Użyj debuggera lub try/catch; sprawdź dane wejściowe
Logiczny (Logic)	Program daje złe wyniki	Tylko debugger / tester	Zły warunek pętli, błędna formuła, off-by-one	Śledź przepływ danych debuggerem; testuj edge cases

Przykłady w Pythonie:

Błąd składniowy — SyntaxError:

```
if x > 5 # brakuje dwukropka!
    print("duże")

# SyntaxError: expected ':'
# Interpreter NIE uruchomił żadnej linii
```

Błąd czasu wykonania — ZeroDivisionError:

```
liczba = int(input("Podaj liczbę: ")) # użytkownik wpisał 0
wynik = 100 / liczba # ZeroDivisionError!

# ZeroDivisionError: division by zero
```

Błąd logiczny — off-by-one:

```
# Chcemy zsumować liczby 1..5, ale pętla kończy się na 4
suma = 0
for i in range(1, 5): # ← powinno być range(1, 6)!
    suma += i

print(suma) # wypisuje 10, a powinno 15
# Brak błędu! Program działa — ale daje ZŁĄ odpowiedź.
```

Metodologia debugowania — algorytm pracy

1

Przeczytaj komunikat błędu w całości

Typ błędu → plik → numer linii → opis. Nie scrolluj od razu do StackOverflow.

2

Zreprodukuje błąd

Upewnij się, że możesz wywołać błąd konsekwentnie. Błąd, którego nie możesz odtworzyć, jest bardzo trudny do naprawienia.

3

Izoluj problem

Ustaw breakpoint przed błędem. Zawęż obszar — komentuj sekcje kodu, sprawdź czy błąd nadal występuje.

4

Śledź wartości zmiennych

Użyj watch expressions lub krokowania. Sprawdź, w którym dokładnie kroku wartość "idzie nie tak".

5

Popraw i przetestuj

Wprowadź poprawkę, uruchom WSZYSTKIE testy. Sprawdź edge cases.

6

Zrozum, dlaczego był błąd

Najważniejszy krok. Co doprowadziło do błędu? Czy można zapobiec podobnym w przyszłości?

Popularne debuggery wg języka:

Język	Debugger	IDE / narzędzie	Komenda
Python	pdb (Python Debugger)	VS Code, PyCharm	<code>python -m pdb skrypt.py</code>
JavaScript	Chrome DevTools, Node Inspector	VS Code, Browser	F12 w przeglądarce
C / C++	GDB (GNU Debugger)	VS Code, CLion	<code>gdb ./program</code>
Java	JDK	IntelliJ IDEA, Eclipse	Przycisk Debug w IDE
C#	Wbudowany w .NET	Visual Studio	F5 (Debug mode)

Schemat decyzji:

Chcę szybkiego, gotowego pliku do wdrożenia? → Kompilator (C++, Rust, Go)

Chcę szybko przetestować pomysł? → Interpreter (Python, Node.js, Ruby)

Chcę przenośności i szybkości? → Hybryd JIT (Java, C#, JavaScript V8)

Mam błąd, który nie jest składniowy? → Debugger zawsze

Nie rozumiem co robi kod? → Debugger z krokowaniem (step by step)

Podsumowanie lekcji

Kompilator	Interpreter	Debugger
Tłumaczy CAŁY kod przed uruchomieniem	Wykonuje KOD LINIA PO LINII w czasie rzeczywistym	KONTROLOWANE wykonanie — krok po kroku
Produkuje plik wykonywalny (.exe, ELF...)	Nie tworzy pliku wykonywalnego	Pokazuje stan zmiennych i call stack
Wykrywa błędy ZANIM program dotrze do użytkownika	Błędy pojawiają się dopiero w trakcie działania	Jedyne narzędzie do wykrycia błędów LOGICZNYCH
C, C++, Rust, Go	Python, Ruby, PHP, Bash	GDB, pdb, Chrome DevTools, VS Code

Hybrydy (JIT):

Java i C# kompilują do bytecode, a JVM/CLR używa kompilacji Just-In-Time. JavaScript V8 zaczyna jako interpreter, ale gorący kod kompiluje w locie. Python tworzy pliki .pyc (bytecode cache).

Trzy rodzaje błędów:

Składniowy — wykrywa kompilator/interpreter przed uruchomieniem

Runtime — pojawia się podczas działania (dzielenie przez zero, null ptr)

Logiczny — program działa, ale daje złe wyniki — tylko debugger pomoże!